VOLLEYBALL HIGHLIGHTS · EXPLAINER

How the volleyball highlights tool works

A plain-language tour of the whole project, with the AI and machine-learning parts explained in depth: which models do what, how they talk to each other, how the tool learns a group's taste, and how fine-tuning Qwen would work.

The one-paragraph version

You point the tool at a folder of game footage. It listens (Whisper writes down what everyone says; PANNs hears laughter, cheering and shouting), looks (a vision-language model, Qwen3-VL, watches 24-second stretches and describes what happens), and reads (Qwen rates how unhinged the things people said were). A scoring step combines all of that into "moments", favouring long rallies and big spikes, and the best ones are cut from the original 4K files into a ~20 minute reel. Every moment and every correction you make is saved in a library, and three small learners use it to adapt to your group over time. A fourth learner, retraining Qwen itself (Phase 7), is prepared but not built yet.

Everything runs locally on one Windows PC with an NVIDIA graphics card (built and tested on a 16 GB card). No cloud services, no paid APIs, and nothing is uploaded anywhere. Numbers quoted here (weights, thresholds, sizes) are the defaults from `config.example.yaml` and the code.

Contents

1. The big picture
2. AI basics in five minutes
3. The ears: Whisper and PANNs
4. The cheap filter: motion
5. The eyes: Qwen3-VL watching the game
6. Reading the room: unhinged comments and "clip that"
7. Fusing it all into scored moments
8. Choosing and rendering the reel
9. Nametags: YOLO and ByteTrack
10. How the tool learns
11. Fine-tuning Qwen (Phase 7)
12. Hardware, speed, the app and where everything lives

13. Glossary

Status marks

BUILT code exists and works PLANNED designed but not written yet AI MODEL a trained neural network does this step

The cast of models

Model	What kind of AI	Its job here	Runs on
Whisper large-v3-turbo via faster-whisper	Speech recognition (transformer)	Writes down every word, with the time each word was said	GPU
PANNs Cnn14	Sound classifier (convolutional network)	Scores every second for laughter, cheering, shouting and wind	GPU
Qwen3-VL 8B instruct via Ollama	Vision-language model (multimodal LLM)	Watches each 24 s stretch and labels it; also reads the transcript for unhinged comments	GPU
SigLIP base	Image embedding model	Gives every second of footage a "fingerprint" so similar past moments can be found	GPU
Ranker: logistic regression, then LightGBM	Classic machine learning, trained on your PC from your feedback	Learns which moments your group actually wants in the reel	CPU, seconds
YOLO11s + ByteTrack	Object detector + tracker	Finds and follows each player for the nametags	GPU

Only the ranker is trained by this project. The others are downloaded pre-trained and used as they are. Phase 7 would make Qwen the second model trained on your data.

1 · The big picture

The tool is one Python program, `make_highlights.py`, that takes a session folder (or one video) and produces `output/<session>/<session> highlights.mp4`, plus a separate clip each time someone said "clip that". You can drive it from the command line or from the local browser app (`--gui`). Several session folders can be queued and run one after another, each kept as its own session.

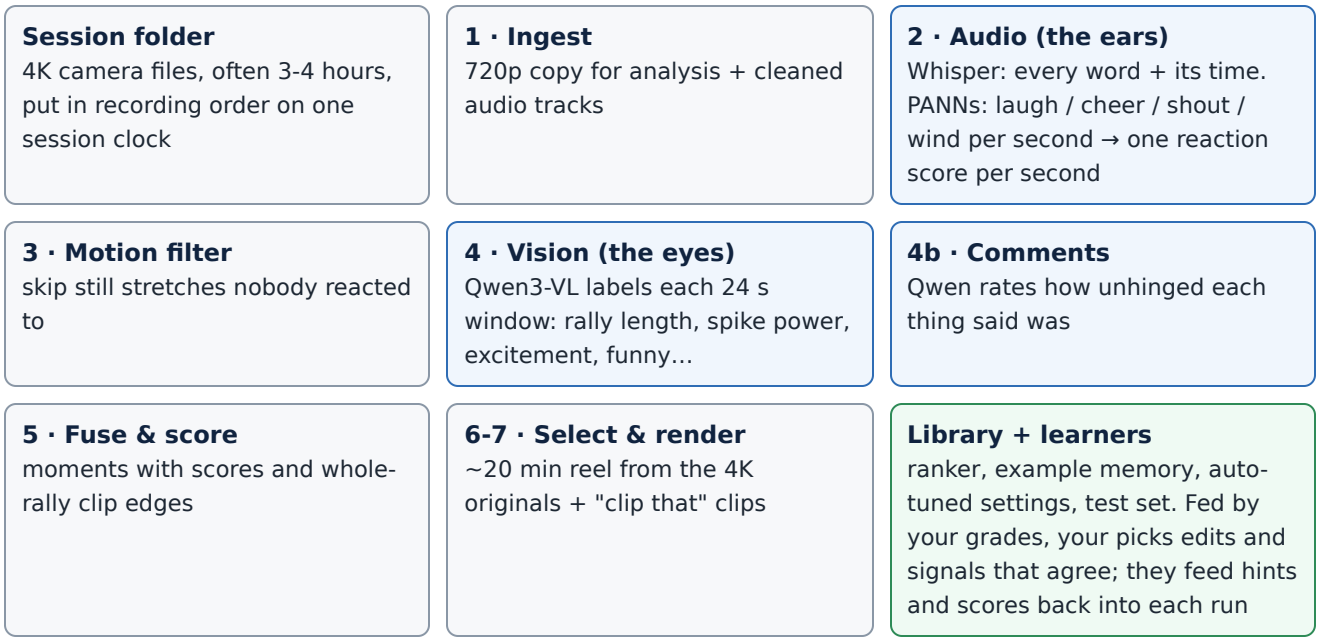


Figure 1. The pipeline. Blue boxes are steps where a neural network does the work; green is the learning loop.

Design rules that shape everything

- **Local only.** Every model runs on your PC.
- **One model in memory at a time.** 16 GB of video memory is plenty for any single model, but not for all of them at once. Each stage loads its model, uses it, and frees it before the next stage starts.
- **Everything is cached.** Each stage saves its results in `work/` with a fingerprint of the settings that produced them. Change a scoring weight and only the scoring re-runs, in seconds. Change the Qwen prompt and only the vision tagging re-runs. Stop a run halfway and the next run picks up where it left off.
- **Raw model output is kept separate from scoring.** PANNs stores all 527 sound probabilities for every second; deciding which of those count as "laughter" happens later and can be changed without re-running PANNs.
- **Analysis uses a 720p copy; the output uses the originals.** The models never need 4K, and the small copy is much faster to read. The final clips are cut from the original files, so the reel keeps the camera's full resolution.
- **Settings live in one YAML file.** Tuning never needs a code change, and the tool never rewrites your config file. Your own `config.yaml` (with your players' names) stays on your

PC; `config.example.yaml` is the shared starting point.

Why several models instead of one?

No single model is good at everything here. Whisper is excellent at words but deaf to laughter. PANNs hears laughter but can't tell you what was funny. Qwen can see a spike, but at the speed it runs (a few seconds per 24-second window) it can't look at every frame, and it only "hears" through the text the other models give it. Combining them works better than any one alone, and the audio models also tell Qwen where to look more closely.

2 · AI basics in five minutes

A few ideas come up again and again in this document. Here they are once, simply.

A model is a big function with learned numbers

A neural network takes numbers in (pixels, audio samples, words turned into numbers) and gives numbers out (probabilities, a label, the next word). In between are millions or billions of adjustable numbers called parameters or weights. "Training" means showing the model many examples and nudging those weights so its answers get closer to the right ones. "Inference" means using the finished model. This tool does inference with every model, and training only for the small ranker (and, later, Qwen).

Pre-trained models

Training a model like Whisper took a big lab, huge amounts of data and a lot of computing. We just download the result. The skill is in choosing the right pre-trained model for each job and feeding it the right inputs.

Large language models and vision-language models

A large language model (LLM) works on tokens, chunks of words. Given the text so far, it predicts the most likely next token, adds it, and repeats. Done by a model with billions of parameters, that loop is enough to follow instructions, summarise and answer in a required format.

A vision-language model (VLM) like Qwen3-VL adds a vision encoder in front: each image is cut into small square patches, and each patch becomes a vector the language model treats like extra tokens. A frame of video becomes, roughly, a few hundred "picture words" sitting in the prompt next to the text. The model then reads the pictures and the text together and writes its answer.

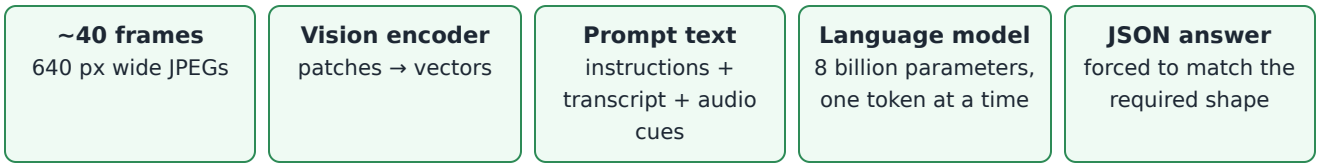


Figure 2. What happens inside Qwen3-VL for one 24-second window.

Context window

A model can only look at so many tokens at once: its context window. Here it is 16,384 tokens (`vlm.num_ctx`). The frames take most of that space, which is why the tool sends at most 40 frames per window at 640 pixels wide.

Quantization

Each of Qwen's 8 billion parameters would normally take 2 bytes (16 GB in total, too big). Ollama's version stores them in roughly 4 to 5 bits each instead. That is why the download is

about 6 GB and fits on a 16 GB card, with a small loss in quality. On an 8 GB card, the 4B model is the better choice.

Embeddings

An embedding is a list of numbers that describes something (an image, a sentence) so that similar things get similar lists. The example memory (chapter 10) uses SigLIP embeddings to find past moments that look like the current one.

Prompting vs fine-tuning

Prompting changes the instructions the model reads; nothing inside the model changes, and the effect is instant. Fine-tuning changes the model's weights by training it on examples of the answers you want; it is slower and needs data, but it can teach things that are hard to put into words. Today the tool only prompts Qwen (including showing it past examples). Phase 7 adds fine-tuning.

Measuring "better": AUC

The learners need a fair way to tell whether a new version is better. The tool mostly uses AUC: pick one moment you said was worth showing and one you said wasn't; AUC is the chance that the scoring ranks the good one higher. 0.5 means coin-flip, 1.0 means perfect. It suits this job because the reel is about ranking moments, not about any exact score.

3 · The ears: Whisper and PANNs

Friends' reactions are the best clue that something happened, for great plays and bloopers alike. So the audio stage runs first, and its results guide everything after it. BUILT AI MODEL

Preparing the sound

Ingest pulls two mono audio tracks out of each video: 16 kHz for Whisper and 32 kHz for PANNs. Outdoor footage is full of wind, so the analysis audio goes through two ffmpeg filters first: a high-pass filter at 120 Hz (cuts the low rumble wind makes) and `afftdn`, a gentle noise reducer. This cleaning is only for analysis; the reel keeps the original sound.

Whisper: what was said, and exactly when

Whisper (by OpenAI, free to download) is a speech-recognition transformer. It turns the audio into a spectrogram (a picture of which pitches are loud at each moment), and an encoder-decoder network writes the words. The tool uses large-v3-turbo (about 800 million parameters, several times faster than large-v3 with similar accuracy) through faster-whisper, which runs it efficiently on the GPU in 16-bit numbers. Settings that matter for volleyball footage:

- **Voice activity detection** (`vad_filter`): a small speech detector skips stretches with no talking. Without it, Whisper tends to "hear" words in wind noise.
- **Word timestamps:** every word gets a start and end time. This is what lets "clip that" and the unhinged-comment finder point at exact seconds.
- **No conditioning on previous text:** in noisy audio, Whisper's habit of using what it just wrote as context can make it repeat itself in loops, so it's turned off.
- **Beam search of 5:** Whisper considers 5 candidate transcriptions at each step and keeps the best.

Output: `work/<file>/transcript.json` (and a readable `transcript.txt`).

PANNs: what kind of sound it was

PANNs ("Pretrained Audio Neural Networks") is a sound classifier. The Cnn14 model is a convolutional network (about 80 million parameters) trained on AudioSet, Google's collection of about two million 10-second clips labelled with 527 kinds of sound. Given a snippet of audio, it gives a probability for every one of the 527 classes at once.

The tool feeds it a 2-second window centred on every second of the game, so each second gets 527 probabilities, saved raw (`panns.npz`). Then they're grouped into four signals by taking the highest probability within each group:

Signal	AudioSet classes in the group
laugh	Laughter, Giggle, Snicker, Belly laugh, Chuckle/chortle, Baby laughter
cheer	Cheering, Applause, Clapping, Crowd
shout	Shout, Yell, Screaming, Children shouting, Battle cry, Bellow, Whoop, Gasp, Groan
wind	Wind, Wind noise (microphone). Not a reaction: used to discount loudness spikes that are just gusts

Each reaction signal is rescaled so the game's strongest moments reach about 1.0, with a floor: a raw probability below 0.3 never counts as "strong", so a game with no real laughter doesn't get its background noise blown up to look like laughter.

Three cheap signals that need no extra model

- **Loudness:** the loudest 100 ms in each second, compared with the median of the surrounding minute. 12 dB above normal scores 1.0, multiplied by (1 − wind probability) so gusts don't count.
- **Keywords:** phrases from the transcript such as "oh my god", "no way", "let's go", "what was that".
- **Speech burst:** how many words are spoken in the 4 seconds after each second, compared with the game's normal rate.

```
reaction = (1.0·laugh + 0.8·cheer + 0.7·shout + 0.5·loudness + 0.4·keyword +
0.3·speech_burst) ÷ 3.7
then smoothed with a 3-second moving average
```

Every signal is saved in `timeline.csv` (one row per second). The top 40 peaks, at least 20 seconds apart, go in `peaks.csv`. They are used three ways: to tell Qwen where to look closely, to create "reaction" moments Qwen missed (often off-court jokes), and to decide how long a clip keeps running after a play.

Keeping sound and picture in sync

Some cameras (DJI ones included) drop short bits of audio now and then. The file's timestamps still say where the sound belongs, but a plain audio extraction closes the gaps, so everything after a gap slides earlier, by several seconds by the end of a long file. That would put Whisper's words, PANNs' laughter and the "clip that" times in the wrong place. The fix is an ffmpeg resampling option (`aresample async`) that fills each gap with silence instead. It's used both when audio is extracted and when clips are rendered, and keeps the sound in line with the picture to within a few hundredths of a second.

4 · The cheap filter: motion

Qwen is the slowest part of the pipeline, so it shouldn't waste time on water breaks. A simple, non-AI movement check decides which stretches are worth showing it. **BUILT**

The 720p copy is read at 4 frames per second, shrunk to a tiny 160×90 grayscale picture, and each frame is compared with the one before. The average pixel change is that second's motion score. A whole game takes a minute or two.

A 24-second window is marked idle and skipped when both are true:

- its movement (75th percentile) is below 35% of the game's typical movement, and
- nobody reacted: the audio reaction never reaches 0.12.

The second rule matters: a joke during a break still has people laughing, so it still goes to Qwen. Idle windows are labelled "downtime" automatically.

> This is classic computer vision (frame differencing), the same idea as a basic motion-detecting security camera. It can't tell a rally from someone walking, which is fine: it only has to rule out stretches that are clearly empty.

5 · The eyes: Qwen3-VL watching the game

This is the heart of the AI. A vision-language model looks at frames from each stretch of the game, reads what was said and heard, and answers a fixed set of questions in a strict format. **BUILT** **AI MODEL**

The model and how it's served

Qwen3-VL 8B is an open vision-language model from Alibaba's Qwen team. It runs inside Ollama, a free local program that loads models onto the GPU and answers requests over `http://localhost:11434`. The tool sends Ollama the pictures and the prompt; Ollama sends back the answer.

The config uses `qwen3-vl:8b-instruct`, not plain `qwen3-vl:8b`. The plain tag is the "thinking" variant, which writes out long reasoning before answering; with 40 images it can run past the 5-minute time limit per window. The instruct variant answers directly, in a few seconds per window on a modern card. Before the first window the tool checks how much of the model landed on the GPU, and warns if part of it is running on the much slower CPU. After the stage the model is unloaded so the next one has the memory.

Cutting the game into windows

Each file is cut into 24-second windows every 16 seconds, so neighbouring windows overlap by 8 seconds and a play near the edge of one window is fully inside the next. A 45-minute file gives about 170 windows, fewer after the motion filter.

Choosing which frames to show

Qwen gets still frames, not video: 1 frame per second across the window as a baseline, plus a burst of 4 frames per second over the 3 seconds before each of the window's two strongest audio reaction peaks. People react after a play, so the seconds before a shout or burst of laughter are where the spike or whiff happened. At most 40 frames are sent (evenly thinned if there are more), each 640 pixels wide, with the exact time of each frame listed in the prompt.

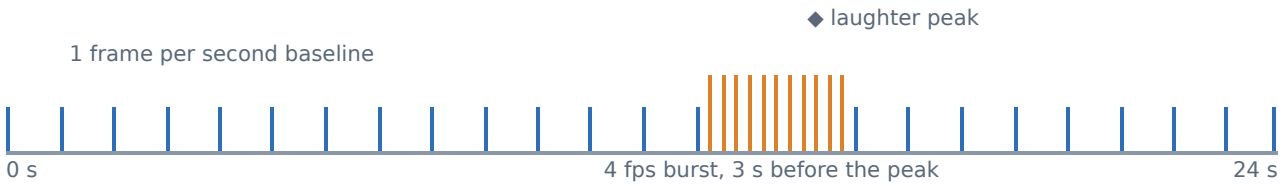


Figure 3. Frame selection for one window: blue ticks are the 1 fps baseline, orange ticks the dense burst before a reaction peak.

What the prompt tells Qwen

The prompt (`prompts/tag_window.txt`) is a template filled in for every window with:

1. What the friends want: long rallies and big spikes first, funny moments and banter after.
2. The frame times.

3. What was said in that stretch, from the Whisper transcript.
4. What the audio picked up, e.g. "+12.5s: strong laughter", ending with "Laughter in this stretch: none heard / a little / clear laughter".
5. (Once there's enough history) similar moments from earlier games and how the group judged them (the example memory, chapter 10).
6. Precise instructions and scoring scales for each field.

This is how the ears feed the eyes: Qwen can't hear, but it reads what Whisper and PANNs heard. Qwen is never told who is who; it tells players apart by what they wear ("the player in the black shirt").

The answer: a strict form to fill in

Field	Meaning
description	One or two plain sentences about what's actually visible. Written first, so the model looks before it judges.
play_active	Is a rally in progress at any point?
rally_start_s, rally_end_s	When the longest rally starts (the serve) and ends (the ball dies), in seconds into the window
attack 0-10	The hardest hit. 7+ only when the jump and a hard downward hit are both visible
event_type	One of: great_play, blooper, celebration, chaos, conversation, rally, downtime
players	Who's involved, described by clothing ("white tank top"); empty if unsure
excitement 0-10	How impressive or intense, with anchors (2-3 short ordinary rally … 8-10 big spike or very long rally)
funny 0-10	6+ only with heard laughter or something clearly comical
peak_offset_s	When the notable moment happens (the hit, not the reaction)

Two mechanisms make sure the answer is usable:

- **Constrained output.** The tool sends Ollama a JSON schema listing every field, its type and order. Ollama then only lets the model produce tokens that keep the answer valid JSON of that shape. Qwen cannot answer in prose or skip a field.
- **Validation.** The answer is checked with pydantic: an attack of 12 or an unknown label is rejected.

Temperature is 0.1 (almost no randomness), so the same window gives nearly the same answer every time.

Lessons from prompt engineering

An early version of the prompt made Qwen call most ordinary point endings (ball out, ball in the net) "bloopers", give nearly everything the same excitement score, and claim people laughed when nobody did. The fixes were all in the prompt, which is the cheapest way to change a model's behaviour:

- "Every rally ends with someone missing… That alone is an ordinary rally ending: label it rally, not blooper."
- "Never assume a reaction", plus the explicit "Laughter in this stretch: none heard" line.
- Concrete score anchors for each number, and "most stretches are ordinary… with low scores".
- Description first, label second, so the model commits to what it sees before choosing a category.

Because the prompt text is part of each window's cache fingerprint, any prompt change automatically re-tags every window on the next run.

Everything Qwen saw is kept

For every window, the cache keeps the exact prompt Qwen received, its answer, which frames it saw and which past examples it was shown. The library also keeps the frame images. This is what makes fine-tuning possible later (chapter 11): the training data is "these exact inputs, and here's the answer it should have given".

6 · Reading the room: unhinged comments and "clip that"

Unhinged comments BUILT AI MODEL

The wild things people say are part of the fun. After the vision stage, Qwen is used a second time, as a plain text model with no pictures, to read the transcript and rate how unhinged each comment is.

1. The Whisper transcript is read in stretches of 30 lines, overlapping by 6, so a comment and the reaction to it are always read together.
2. Each line carries its number, its time, what was said, and a note of what PANNs heard right after it, e.g. `[laughter]` or `[shouting]`.
3. The prompt (`prompts/rate_comments.txt`) explains what counts as unhinged (outrageous trash talk, absurd excuses, fake threats…) and what doesn't (calling the score, "my bad", or Whisper's typical mistakes like "Thanks for watching").
4. It also teaches the key trick: a reaction like "whaaat?!", "bro what" or "you can't say that" usually answers something said just before. Qwen should find and rate that earlier line, not the reaction.
5. Qwen answers with a list: line number, the exact quote, the reaction line, a few words on why, and a 0-10 score. Anything it quotes that isn't actually in the transcript is thrown away (a guard against invented quotes).

```
+2 when a reaction phrase matches within 8 seconds after the comment
   (regular expressions in the config: "whaaat", "what?!", "bro what", "excuse
me", "no shot", "bruh", "that's insane"…)
+1 when clear laughter follows
```

A comment at 5 or more (after boosts) earns a place in the reel: it either widens the clip of the play it was said during, so the comment and the reaction are both in, or becomes a moment of its own. Up to 30% of the reel's length is reserved for them, most unhinged first. Every rated comment is listed in `work/<session>/comments.csv`.

> Why the same model? Qwen3-VL is also a capable text model, so this needs no extra download and no extra GPU memory. Text-only requests are fast compared with the image-heavy vision stage.

"Clip that" BUILT

This one is deliberately not AI. Regular expressions look through Whisper's words for "clip that", "clip it", "clip this", "that's a clip" and "clip-worthy". When one is found:

- The clip covers the best tagged moment (not a conversation) that peaked in the 25 seconds before the words. If there isn't one, it's simply the 15 seconds before.

- It runs 4 seconds past the words, so the "clip that!" itself is in it. Maximum 45 seconds.
- "Clip that! Clip that!" within 20 seconds counts once.
- It's saved as its own landscape video, `<session> clip that N.mp4`, and that moment also gets +0.3 toward making the reel.

> A simple pattern match is the right tool here: the phrase is fixed, the timing comes free from Whisper's word timestamps, and it's instant and predictable.

7 · Fusing it all into scored moments

Fusion turns many overlapping opinions (Qwen's tags, audio peaks, rated comments, "clip that" calls) into one list of distinct moments, each with a score and a clip. This step is arithmetic, not AI, which makes it fast to re-run and easy to tune. **BUILT**

Where moments come from

- Every window Qwen tagged that isn't downtime.
- Strong audio reactions (0.3 or more) with no tagged moment within 10 seconds, often jokes between points. They score 0.8× because the vision model didn't confirm them.
- Unhinged comments said away from any other moment.

Because windows overlap, the same play often appears twice. Moments within 10 seconds of each other (or in the same rally) merge into one, keeping the best-scoring version.

```
score = 0.3 × rally + 0.3 × attack + 0.2 × excitement + 0.1 × fun + 0.1 × reaction
      + label bonus (great play 0.15; blooper, chaos, celebration 0.05)
      + 0.4 × unhinged/10   (if an unhinged comment is in it)
      + 0.3                 (if someone said "clip that" about it)
```

Part	From	How it becomes 0-1
rally	Qwen's rally start/end, joined across overlapping windows	0 for a rally of 5 s or less, rising to 1 at 20 s or more
attack	Qwen	attack ÷ 10
excitement	Qwen	excitement ÷ 10
fun	Qwen + PANNs	0.6 × funny/10 + 0.4 × laughter from 2 s before to 8 s after
reaction	audio timeline	the strongest reaction from 2 s before to 8 s after

The default weights put long rallies and big spikes over everything else. Two worked examples:

A 22-second rally ending in a big spike	A short whiff everyone laughs at
rally 1.0 × 0.3 = 0.30	rally 3 s → 0
attack 8 → 0.24	attack 0 → 0
excitement 7 → 0.14	excitement 3 → 0.06
fun (funny 2, little laughter) → 0.02	fun (funny 8, big laughter) → 0.08
reaction 0.4 → 0.04	reaction 0.8 → 0.08
great_play bonus → 0.15	blooper bonus → 0.05
score ≈ 0.89	**score ≈ 0.27** (just over the 0.25 cutoff)

So with these weights a funny moment makes the reel when there's room after the rallies, or through the unhinged-comment and "clip that" routes. All of this is in the config, so a group that cares more about bloopers can turn the funny weight up.

Where each clip starts and ends

- **Whole rallies** (great play, rally, chaos): from 1.5 s before the serve, through the celebration (at least 4 s after the ball dies, up to 15 s while people are still reacting), stopping 0.5 s before the next serve. Up to 60 s.
- **Other labels** get a set amount before and after the key instant, e.g. a blooper 6 s before and after, a conversation 2 s before and 16 s after. The clip keeps running while the reaction stays above 0.25 (up to 6 s more). 6-20 s.

Clips never cross into another file, except where the camera split one continuous recording into two files.

Your grades override the AI

If you corrected a moment's label on the Grade screen, your label replaces Qwen's. If you marked it "not worth showing", it's left out of every video. Once the ranker has learned enough (chapter 10), it blends its own judgement into the score. Every moment, with every number behind its score, is written to `work/<session>/moments.csv`.

8 · Choosing and rendering the reel

Picking BUILT

1. The target is 20 minutes (allowed 18-22), but never more than 25% of the footage. Moments below a score of 0.25 never make it.
2. Unhinged comments (5+) go in first, most unhinged first, up to 30% of the target.
3. Then moments are added best score first until the target is reached. A moment that overlaps or nearly touches one already chosen (within 1.5 s) is merged into it, up to 70 s per clip.
4. The chosen clips are put back in game order.

The picks go into `picks.csv`, with a thumbnail for each. Set `keep` to `n`, or change the start and end, and run again: that clip leaves and the next best takes its place, and only changed clips are re-encoded. The app's Picks screen does the same with buttons and drag-to-reorder. Every one of these edits is also a lesson for the learners.

Rendering BUILT

- Each clip is cut from the original file with ffmpeg and encoded with NVENC, the video encoder chip built into NVIDIA cards (much faster than the CPU; the CPU encoder is the fallback).
- Transitions vary: dip to black 50%, cross-dissolve 35%, swipe 15%, each 0.5 s. The choice is fixed per join (the same picks give the same reel), never two swipes in a row. Clips with nametags and camera-split joins always dip to black.
- If one transition can't be made (for example on an audio gap), only that join becomes a straight cut; the rest of the reel keeps its transitions.
- Each clip is cached in `work/`, and the reel is joined by copying the encoded pieces, so re-renders are quick.
- A timestamps file next to each reel, `<session> highlights - timestamps.txt`, lists every clip with its real position in the finished video, e.g. `1:00 - 2:00 Long rally #1`. Titles come from the scores: Big spike, Long rally, the label (Great play, Chaos, Blooper…), or Unhinged comment with the quote.

> Themed shorts (top plays, bloopers…) and 9:16 phone versions exist in the code but are off by default (`shorts.enabled: false`), to focus on one landscape reel.

9 · Nametags: YOLO and ByteTrack

For the first 18 seconds of the reel, each player gets a Minecraft-style floating nametag. Finding and following the players is a job for a detector and a tracker. BUILT AI MODEL

- **YOLO11s** (from Ultralytics, about 9 million parameters, ~20 MB download) is an object detector trained on the COCO dataset's 80 everyday object types. In one pass over a frame it draws a box around every person. It looks at the 720p copy at 10 frames per second, at 1280 px detection size so players on the far side are still found.
- **ByteTrack** links those boxes from frame to frame into tracks: "person 3" in this frame is the same person 3 in the next. It predicts where each person will be and matches the predictions to the new boxes. It also keeps low-confidence boxes in play, so a player half hidden behind the net isn't lost.

You name the people; nothing is guessed

The tool never recognises anyone. People wear different clothes every week, and several often wear the same colour, so there is no preset list of who wears what. After a run, the Who is who screen shows a still from the reel's opening with everyone on screen outlined and numbered, and a name box for each. Names from your list autocomplete; an empty box means no tag. People who walk in later get a card with a close-up and the time they show up.

Following is rules on top of the tracker, not a new model. When the tracker loses someone (behind another player, out of frame) and later starts a "new" track, that track takes the name of the person it most likely is: it has to start within 8 seconds of where that named person was last seen, nearby, the same size, looking the same (colours measured on that person in the same clip), and the two must never have been on screen at the same time. Follows are listed under "Followed automatically", where you can change them. A name is never shown on two people at once.

Placement is simple rules too: above the head by default, flipping below the feet when there's no room or the background behind the head (bright sky, the net) would make it hard to read, with hysteresis so it doesn't flicker. The tags are drawn as PNG images in a blocky pixel font and laid over the video with ffmpeg.

10 · How the tool learns

The pre-trained models know about volleyball and laughter in general, but not about your group's taste. The learning system closes that gap. Grading is optional: normal use of the tool already teaches it. **BUILT**

The library: memory across games

`library/moments.sqlite` is a small database that keeps growing across every session; `--force` never wipes it. It holds every tagged window and moment with all the numbers behind its score, the frames Qwen saw and the exact prompt it got, every label from every source with a trust weight, a picture fingerprint per window, and every version of every learner with its test score, so any version can be switched back on. Because it keeps the frames and numbers, the learners keep working after the big video files are deleted. The library is personal data and is never meant to be shared.

Where labels come from

Source	How it's collected	Trust
Hand grade	The Grade screen: a moment loops (with sound) and you press Y worth showing, N not, F funny, 1-7 to fix the label, arrows to nudge the edges. Quick mode shows about 10 per session, the ones the tool is least sure about.	1.0
Cut	You set keep = n in picks.csv or dropped it on the Picks screen: "not a highlight"	0.6
Trim	You changed a clip's start or end: "worth showing, with these edges"	0.5
Kept	You edited picks.csv but left this one in: a weak "yes"	0.3
Agreement	Made by the tool itself, only when vision, audio and movement all point the same way	0.4

When a moment has several labels, the most trusted kind wins. Agreement labels let the tool teach itself a little, safely: "highlight: yes" only when Qwen called it a great play, blooper, celebration or chaos with excitement or funny of 7+, the audio reaction is loud (0.4+), and movement is 1.2× above normal. Two guardrails stop it learning its own mistakes: these labels can never be more than half the training weight, and any label from you overrides them.

"Least sure first" in quick grading asks about moments where one answer teaches the most: those right at the reel cutoff, ones where the group reacted loudly but Qwen rated them low, or the reverse.

The test set: the referee

About 100 moments you grade with `--grade test` (or on the Test set screen) are flagged gold and never trained on. They're the fixed exam every new version must pass: a new

ranker, new settings or (later) a fine-tuned Qwen is switched on only if it scores better than what's in use. Until there are 20 gold moments, the learners use 5-fold cross-checking instead.

> **Why this matters**
> A system that teaches itself can drift and get worse without anyone noticing. A fixed exam it never studies from is the standard defence. Every version is kept, so `--use-ranker` and `--use-settings` can switch back at any time.

Learner 1: the ranker

Two small models answer two questions about each moment: is it worth showing? and is it funny? They read 30 numbers per moment: Qwen's excitement, funny, attack and label; rally length; unhinged score; laughter, cheering, shouting, loudness, keywords and speech burst around it; wind; movement; clip length and number of players involved.

- **Up to 400 labels: logistic regression.** In effect a learned version of the hand-set weights: it finds how much each of the 30 numbers should count, with a little regularisation and balanced classes.
- **From 400 labels: LightGBM,** a gradient-boosted decision-tree model. It builds 200 small trees, each correcting the previous ones' mistakes, and can learn combinations like "laughter only matters when the rally was short".

Blending in gradually. The ranker starts after 30 labelled moments (at least 5 yes and 5 no). Its say grows with the number of labels: final score = (1 − a) × config score + a × ranker probability, where a = your labels ÷ 300. It retrains automatically after every run, in seconds on the CPU. No single session can count for more than twice a typical one, so one unusual game can't take over.

Learner 2: the example memory

This teaches Qwen your group's standards without retraining it, by showing it examples in the prompt (few-shot or retrieval-augmented prompting).

1. SigLIP (a Google image model, about 800 MB) turns every second of footage into an embedding.
2. The average look of each file (court, light, camera angle) is subtracted, so what's left is what makes a stretch different: a player on the ground, everyone at the net, a group hug.
3. A window's fingerprint is the average over its seconds.
4. When Qwen tags a new window, the tool finds the 4 most similar past moments (from other sessions, only ones you graded, cut or trimmed, never the test set) and adds them to the prompt: what was first said about each, and how the group judged it.

It switches on once there are 20 such examples, roughly from the third game.

Learner 3: auto-tuned settings

Because every moment keeps the numbers its score was made from, the tool can recompute every past score for any settings instantly. The tuner searches how excitement, funny and reaction share their weight, the funny/laughter mix, the bonus per label and how much audio-only moments count, and keeps whichever settings best rank your "yes" moments above your "no" moments. Clip lengths per label are learned from your trims, and the reel cutoff is set just under 90% of the moments you said were worth showing. It starts at 40 labelled moments and, like the ranker, must beat the current settings on the test set. The config file itself never changes, and if you edit a value after it was tuned, your edit wins.

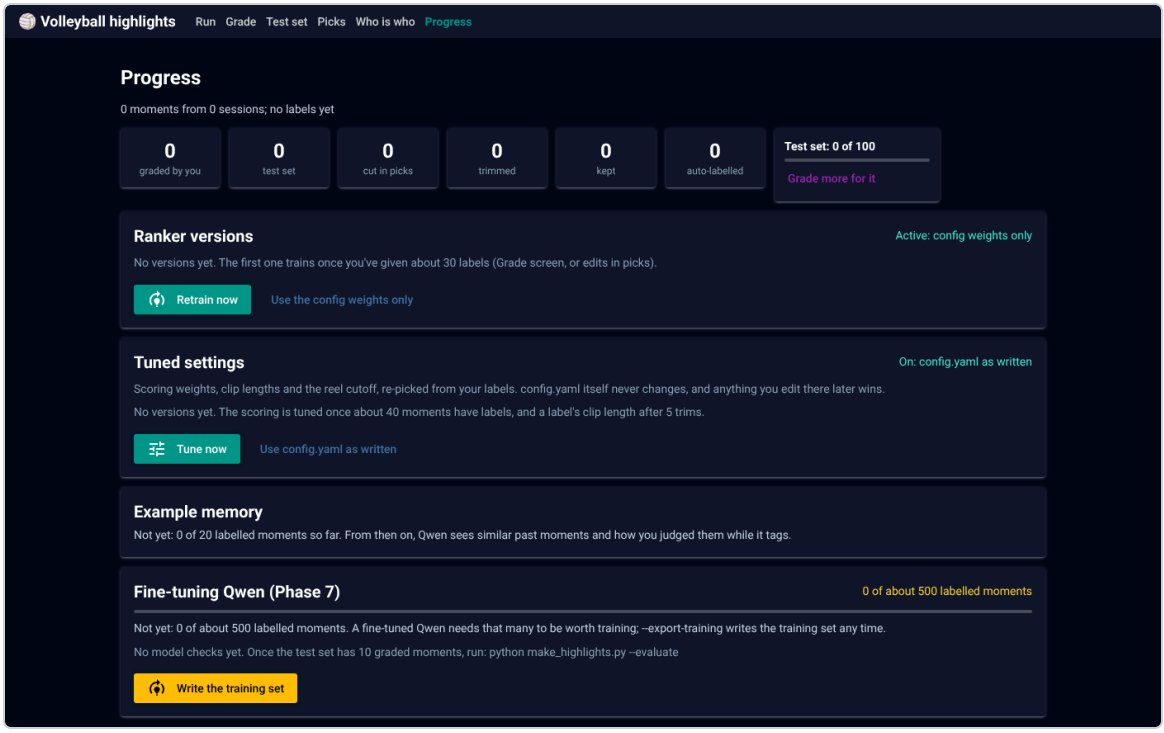


Figure 4. The app's Progress screen on a fresh install: label counts, ranker versions, tuned settings, the example memory and Phase 7 readiness.

11 · Fine-tuning Qwen (Phase 7)

The learners above work around Qwen: they re-weight its answers and give it examples. Fine-tuning would change Qwen itself, so it gets better at what it literally sees, like telling a whiff from a set or recognising a dive. The groundwork is built; the training itself is not.

PREP BUILT TRAINING PLANNED

`--export-training`: the training set BUILT

Writes `library/training/<date>/` with `train.jsonl`, `val.jsonl`, `test.jsonl` and a `manifest.json`. Each line is one example in the chat format Unsloth's vision fine-tuning reads: the frames Qwen saw, the exact prompt it got, and the answer it should have given. That target answer starts from what Qwen said and applies your corrections:

- event_type: the label you set while grading.
- excitement: raised to at least 6 if you said worth showing; lowered to at most 3 if not.
- funny: at least 6 if you marked it funny; at most 3 if not.
- play_active: false only for downtime.
- Description, players and timing stay as Qwen wrote them.

Only trusted labels count (grades, cuts and trims), each example weighted by its most trusted label. About 1 in 10 moments is held back for validation, and no window that shows a test-set moment is included, so the fine-tuned model can be judged fairly.

```
{"messages": [
  {"role": "user", "content": [
     {"type": "image", "image": "frames/…/00.jpg"}, … ~40 frames …,
     {"type": "text", "text": "You are watching a casual outdoor volleyball game…
(the exact prompt)"}]},
  {"role": "assistant", "content": [
     {"type": "text", "text": "{\"description\": \"…\", \"event_type\": \"rally\",
\"excitement\": 2, …}"}]}],
 "weight": 1.0}
```

`--evaluate`: the model check BUILT

Re-tags the windows behind your test-set moments with any model(s) you name, e.g. `--evaluate qwen3-vl:4b-instruct qwen3-vl:8b-instruct`, and compares them: share of moments given the label you set, AUC of excitement against your worth-showing grades, AUC of funny against your funny marks, and the average of the three. The original answers are scored too, so a new model or an edited prompt can be compared without re-running the old one. It needs at least 10 graded test-set moments; results go to `library/evals/`.

Readiness

The Progress screen and `--check` show how close the library is: fine-tuning is considered worth trying at about 500 trusted labelled moments outside the test set

(`learning.finetune.min_labels`), roughly 5-10 sessions of normal use with some grading.

The plan for the training itself PLANNED

LoRA and QLoRA, simply. Retraining all 8 billion parameters would need far more than 16 GB of video memory. LoRA (Low-Rank Adaptation) keeps the original weights frozen and trains small extra matrices added alongside some layers. The result is an add-on of tens of megabytes that nudges the model's behaviour. QLoRA also holds the frozen original in 4-bit form during training, which should make an 8B vision model trainable on a 16 GB card (an estimate that needs a test run to confirm).

The tool: Unsloth, a free library that makes LoRA/QLoRA training faster and lighter on memory, and can export to GGUF, the format Ollama loads. The plan is to install it in its own environment (likely WSL2) so its pinned PyTorch can't break Whisper or PANNs.

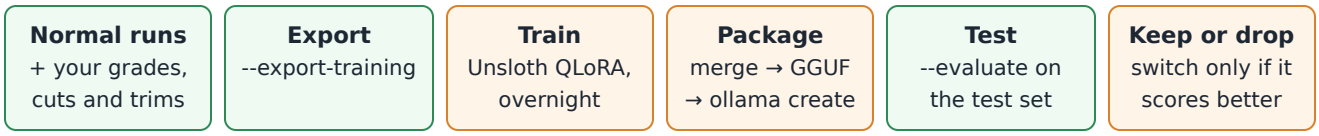


Figure 5. A fine-tune round. Green steps exist today; orange steps are planned.

Open risks

- Ollama may not accept a fine-tuned Qwen3-VL with its vision part intact. The fallback would be llama.cpp or serving it directly in Python; the rest of the pipeline wouldn't change.
- Memory and time estimates for training on a 16 GB card are unconfirmed until a test run.
- Also designed but not built: an overnight "second opinion" pass that re-tags uncertain moments several times, an automatic trigger after enough new labels, and a `--use-model` switch. Today, switching models means editing `vlm.model`.

Realistic expectations

The ranker and example memory should give most of the visible improvement over the first few sessions. Fine-tuning is slower to pay off and mainly sharpens what Qwen recognises in the pictures. The test set will show the real numbers.

12 · Hardware, speed, the app and where everything lives

Step	Hardware	Notes
Ingest (720p copy, audio)	GPU decoder/encoder + CPU	ffmpeg
Whisper	GPU, 16-bit	Falls back to the CPU (much slower) if the GPU libraries are missing
PANNs	GPU	Batches of 32 seconds
Motion	CPU	A minute or two per game
SigLIP fingerprints	GPU	Every second of footage
Qwen3-VL tagging	GPU via Ollama	The slow step: a few seconds per window, so over an hour for a 3-4 hour session
Comments	GPU via Ollama	Text only, quick by comparison
Fuse, select, ranker, tuner	CPU	Seconds
YOLO tracking	GPU	Only the opening seconds that get nametags
Render	GPU (NVENC)	Cut from the original files

A Windows gotcha

Whisper on Windows borrows its CUDA libraries from PyTorch, so a CPU-only PyTorch (which a plain `pip install` can quietly put in) also breaks Whisper's GPU use. Install PyTorch from the CUDA index first, as the README says; `--check` catches the problem and gives the exact fix.

The app

`python make_highlights.py --gui` opens a local browser app (built with NiceGUI) at `localhost:8765`. Screens: **Run** (start, pause, stop, a queue of session folders, live progress and a road map of the stages), **Grade** and **Test set** (the labelling screens, which can play the whole stretch Qwen looked at), **Picks** (edit the reel), **Who is who** (name the players) and **Progress** (each learner and Phase 7 readiness). Each session date keeps its own grading, nametags and videos.

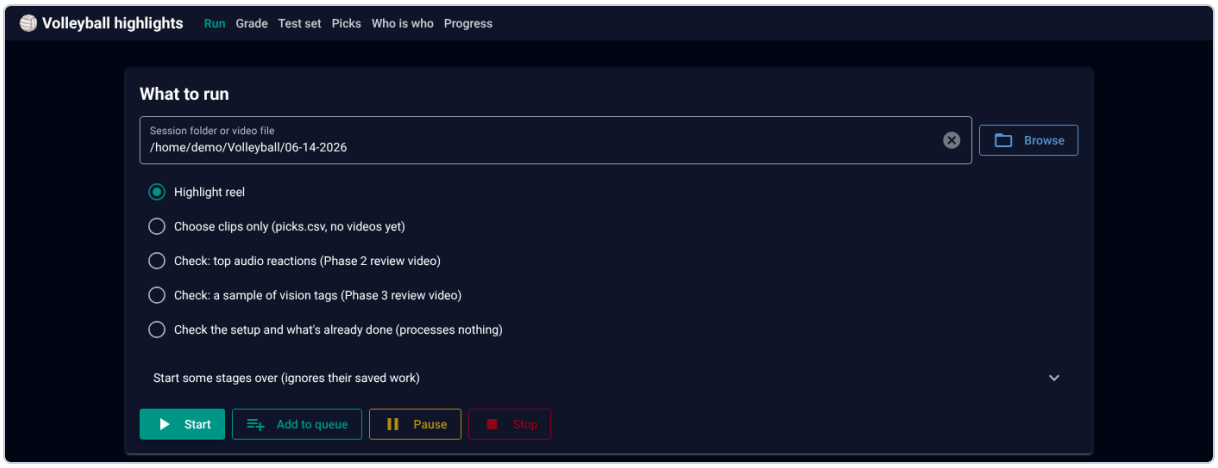


Figure 6. The Run screen: pick a session folder, choose what to make, and start it or add it to the queue.

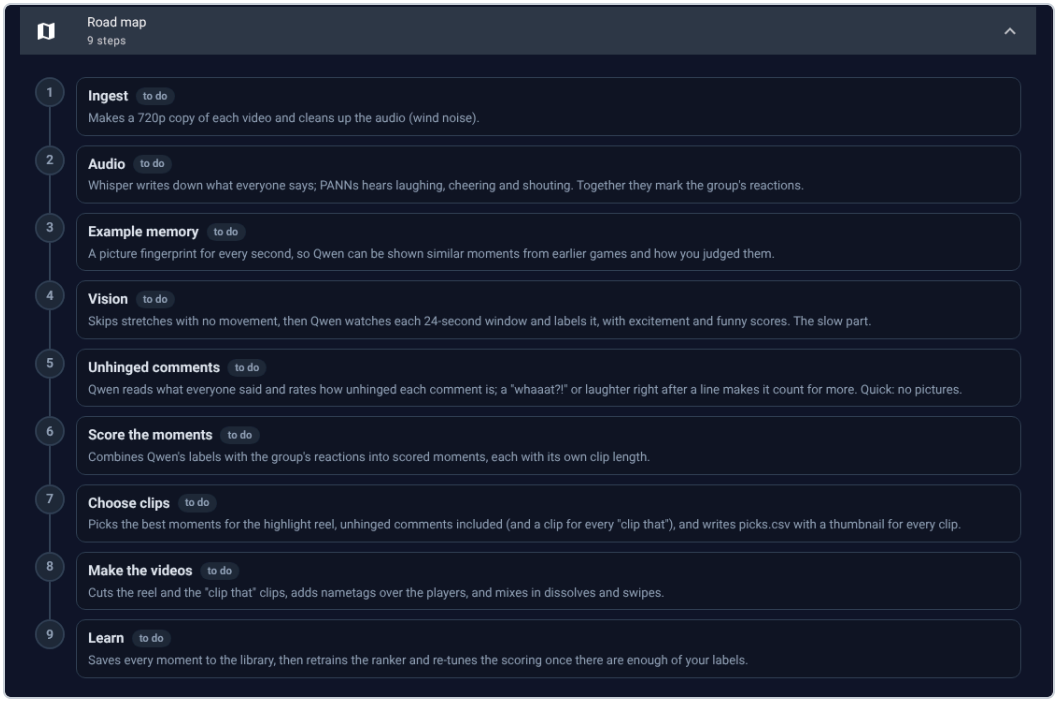


Figure 7. The Run screen's road map, which follows a run through its nine steps.

Folders

Folder	What's in it	Can it go?
library/	The learning memory: moments, labels, frames, fingerprints, model versions, training sets, evals	Always keep (and back up)
models/	Downloaded YOLO and SigLIP	Keep (re-downloads otherwise)
work/<file>/	Per-video caches: 720p copy, audio, transcript, PANNs scores, motion, Qwen answers, clips	720p copy needed until graded; small files worth keeping
work/<session>/	moments.csv, comments.csv, picks history, who-is-who answers, run logs	Small; keep
output/<session>/	The reel, "clip that" clips, picks.csv, thumbnails	Your results
prompts/	The two prompts Qwen reads	Part of the tool

library/, models/, work/, output/ and your `config.yaml` hold your footage, your friends' names and your grades. They are listed in `.gitignore` and never belong in a shared copy of the project.

13 · Glossary

AudioSet
Google's dataset of ~2 million labelled 10-second sound clips in 527 classes; what PANNs was trained on.

AUC
The chance a random "yes" moment scores higher than a random "no" moment. 0.5 = guessing, 1 = perfect.

ByteTrack
A tracker that links detection boxes across frames into one identity per person.

Context window
How many tokens a language model can consider at once (16,384 here).

Cross-check (k-fold)
Splitting labels into k parts and scoring each part with a model trained on the others, so nothing is graded by a model that saw it.

Embedding / fingerprint
A list of numbers describing an image or text, where similar things get similar lists.

Few-shot prompting
Showing a model worked examples inside the prompt so it copies their standard. The example memory does this.

Fine-tuning
Further training a pre-trained model on your own examples so its weights change.

GGUF
The model file format used by llama.cpp and Ollama.

Gold / test set
Hand-graded moments never used for training; the exam every new version must pass.

Inference
Using a trained model to get answers, without changing it.

LightGBM
A fast gradient-boosted decision tree library; the ranker's model from 400 labels.

Logistic regression
A simple model that learns one weight per input number and outputs a probability; the ranker's first model.

LoRA / QLoRA
Training a small add-on beside a frozen model instead of the whole model; QLoRA keeps the frozen part in 4-bit to save memory.

NVENC
The hardware video encoder in NVIDIA cards.

Ollama
A free local program that runs language and vision models and answers requests over HTTP.

PANNs / Cnn14
Pre-trained audio neural networks; Cnn14 is the convolutional sound tagger used here.

Parameters / weights
The learned numbers inside a model. Qwen3-VL 8B has about 8 billion.

Prompt
The text instructions (plus images) given to a language model.

Pydantic
A Python library that checks data has the right fields and types; used to validate Qwen's answers.

Quantization
Storing a model's numbers with fewer bits so it fits in less memory.

Qwen3-VL
An open vision-language model from Alibaba's Qwen team; the tool's "eyes".

SigLIP
A Google image-text model; used here only for picture embeddings.

Spectrogram
A picture of sound: time across, pitch up, brightness = loudness. Both Whisper and PANNs read audio this way.

Temperature
How much randomness a language model uses when picking each token. 0.1 = nearly none.

Token
A chunk of text (or an image patch) as the model sees it.

Unsloth
A free library for fast, memory-light LoRA fine-tuning; the planned tool for Phase 7.

VAD
Voice activity detection: finding where speech is, so the rest can be skipped.

Whisper
OpenAI's open speech-recognition model; faster-whisper runs it efficiently.

YOLO
"You Only Look Once", a family of fast object detectors; YOLO11s finds the players.